Table of Contents

1. Pre-requisites....2

2. Purpose with the Problem scenario....2

3. Introduction....2

4. Detailed Explanation....4

4.1. Content....4

4.1.1. Relating real world scenario using technical Concepts....4

4.1.2. Theory....6

4.1.2.1. Inheritance and Access Modifiers:....6

4.1.2.2. Types of Inheritance:....7

4.1.2.3. Why Java doesn't support Multiple inheritance using classes:....10

4.1.2.4. Super Keyword:....11

4.1.2.5. Instance of operator:....13

4.1.3. Pros & Cons....14

4.1.4. Syntax....15

4.2. Code Snippets....17

4.3. Real Time Usage....17

4.4. Check Yourself....17

5. Error Zone - commonly made mistakes....17

6. Sample code....18

7. Additional Information (Optional)....18

8. Best Practices....19

9. Summary or Take Away Points....19

10. Related Topics: 19

11. References 19

1. Pre-requisites

Should have the basic understanding on Classes and Objects , Object Oriented programming, Constructors

2. Purpose with the Problem scenario

Consider a scenario where an organization has to track the details of its employees. There can be two types of employees – Regular and Contract. All the employees will have some common attributes like EmployeeId, Name, Designation, ContactNumber. The Regular Employees will have Monthly Basic whereas Contract Employees will have Hourly Rate and Number of Hours worked in a month. How the organization is going to model its classes in this case?

There are two approaches – one is to declare all the common attributes and methods in both Regular and Contract Employees classes. This results in duplicating the common attributes and methods in both classes.

The second approach is to have a class Employee with common attributes (& methods). Reuse/Inherit the common attributes in Regular and Contract Employee classes by extending/inheriting the Employee class. The specific attributes and methods can be declared only in the respective classes. Mentioning all the common members in a super class is called generalization , writing the specific functionality in sub classes is called specialization.

The second approach has the advantages : re-usability and readability. Also if any attribute/method applicable to both Regular and Contract employees has to be added in future, the changes can be done in Employee class only.

3. Introduction

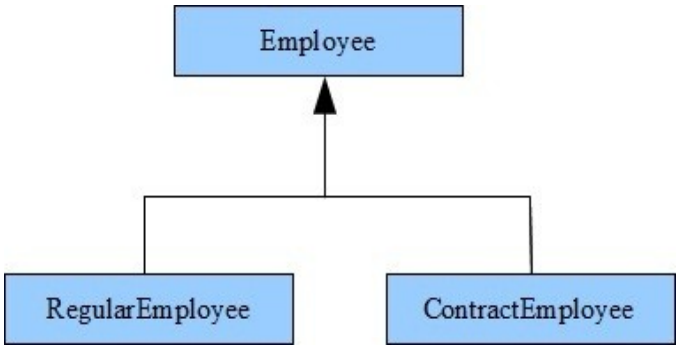
Inheritance is the process of acquiring the properties of one class by another. It allows us to build hierarchies of classes/interfaces. It is also called IS-A relationship between classes/interfaces involved in the hierarchy.

The idea of inheritance is simple but powerful: When you want to create a new class and there is already a class that includes some of the code that you want, you can derive your new class from the existing class. In doing this, you can reuse the fields and methods of the existing class without having to write (and debug!) them yourself.

A subclass inherits all the *members* (fields, methods, and nested classes) from its superclass. Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.

Except java.lang.Object, which has no superclass, every class has one and only one direct superclass (single inheritance). In the absence of any other explicit superclass, every class is implicitly a subclass of Object.

In the scenario mentioned in above section, the classes Regular Employee and Contract Employee are inheriting/extending the class Employee. The inherited class Employee is called Super class or Base class or Parent class or Generic class where as the inheriting class ie Regular Employee or Contract Employee is called Sub class or Child class or Specific class.



Inheritance can be achieved by extending the functionality of a class or implementing the functionality of an interface.

In an inheritance hierarchy, the classes/interfaces at the top of the hierarchy will be more Generic classes and the classes at the bottom of the hierarchy will be more Specialized. The subclasses can specialize themselves in the following ways:

- By re-declaring the attributes present in super class (called Field Hiding)
- By adding new attributes
- By re-defining the instance methods present in super class (called Method Overriding)
- By re-defining the static methods present in super class (called Method hiding)
- By adding new methods

4. Detailed Explanation

4.1. Content

4.1.1. Relating real world scenario using technical Concepts

For the Organization scenario mentioned above, the class Employee will be as follows:

```
class Employee {
        int empId;
        String empName;
        String designation;
        public void display(){
                System.out.println("Employee display method");
        }
}
```

Both Regular and Contract employees have the properties mentioned in Employee above and additionally some specific properties. In this case, we will reuse them by extending the Employee class ie.

```
class RegularEmployee extends Employee {
        int basic;
}
class ContractEmployee extends Employee {
        int hourlyRate;
        double numHours;
}
```

Now the class RegularEmployee will have the attribute basic as well as the attributes inherited from Employee ie. empId, empName, designation. The method display() is also inherited from Employee class. Similar is the case with ContractEmployee.

Let us consider the following code snippet where we try to create Regular and Contract employees:

```
class EmployeeDetails {
        public static void main(String[] args){
                Employee e = new Employee();
                RegularEmployee re1 = new RegularEmployee();  //line 4
                Employee e1 = new RegularEmployee();     //line 5
                ContractEmployee re2 = new ContractEmployee(); //line6
                Employee e2 = new ContractEmployee();    //line 7
        }
}
```

Look at lines 4 and 5 where we are trying to create the object of RegularEmployee. In line 4, re1 which the reference variable of RegularEmployee class is referring to the object of RegularEmployee (***which is the common way we created an object so far***).

In line 5, the reference variable of Employee class is referring to the object of RegularEmployee. This is valid as RegularEmployee class inherits Employee class ie . every **RegularEmployee IS-A Employee** by the process of inheritance.

Similarly every **ContractEmployee IS-A Employee** (line 7).

When we are creating a sub class object, the order of invocation of constructor is that the super class constructor will be invoked first followed by subclass constructor. This can be checked by putting constructor in the above classes.

4.1.2. Theory

4.1.2.1. Inheritance and Access Modifiers:

A subclass inherits all of the *public* and *protected* members of its parent, irrespective of the package the subclass is in. If the subclass is in the same package as its parent, it also inherits the *default* members of the parent. We can use the inherited members as it is, replace them, hide them, or supplement them with new members:

- The inherited fields can be used directly, just like any other fields.
- You can declare a field in the subclass with the same name as the one in the superclass, thus *hiding* it (not recommended). We can access the super class variables by using the super keyword.
- You can declare new fields in the subclass that are not in the superclass.
- The inherited methods can be used directly as they are.
- You can write a new *instance* method in the subclass that has the same signature as the one in the superclass, thus *overriding* it.
- You can write a new *instance* method in the subclass that has same name but different signature from the one in superclass, thus overloading it.
- You can write a new *static* method in the subclass that has the same signature as the one in the superclass, thus *hiding* it. This also called method hiding.
- You can declare new methods in the subclass that are not in the superclass.
- You can write a subclass constructor that invokes the constructor of the superclass, either implicitly or by using the keyword super.
- A subclass does not inherit the private members of its parent class. However, if the superclass has public or protected methods for accessing its private fields, these can also be used by the subclass.
- A nested class has access to all the private members of its enclosing class—both fields and methods. Therefore, a public or protected nested class inherited by a subclass has indirect access to all of the private members of the superclass.
- The final methods or variables used in superclass will be inherited. The final variables can be hidden. The final methods cannot be overridden but they can be overloaded in sub class.
- The default members of a class cannot be inherited outside the package. Hence the scope of default members of a class is only within the package.
- The protected members of a class can be inherited outside the package, but they cannot be accessed to non-sub classes outside the package. Hence the scope of protected members is all classes within the same package & subclasses outside the package.

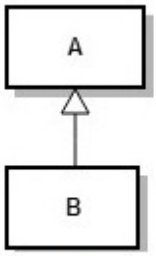
- The public members of a classes can be accessed to all classes outside the package irrespective of whether the class is inherited or not. The scope is to all classes both inside and outside the package.

4.1.2.2. Types of Inheritance:

➢ Depending on the number of classes/interfaces involved in the inheritance, inheritance can be classified as:

◆ **Single inheritance (Single level inheritance)**

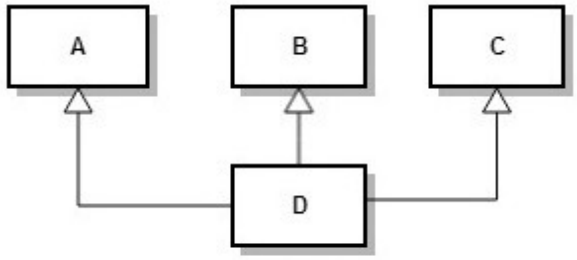
- One class can extend another class (or)
- One interface can extend another interface (or)
- One class can implement an interface.



Single Inheritance

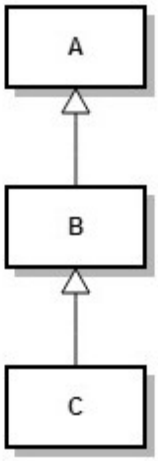
◆ **Multiple inheritance**

- One class can extend multiple classes ***(not supported in Java)***
- One class can implement multiple interfaces
- One class can extend one class and implement one/more interfaces
- One interface can extend multiple interfaces



Multiple Inheritance

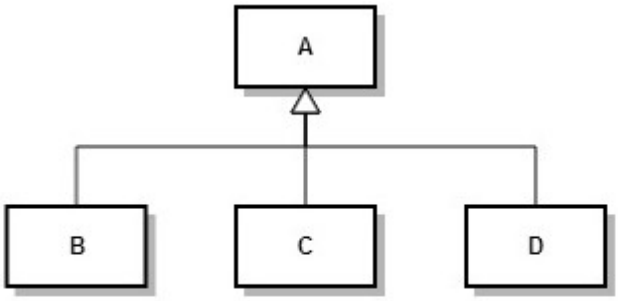
- **Multilevel inheritance**
  - One class can extend a class which in turn extends a class or implements an interface
  - One class can implement an interface which in turn extends one or more interfaces
  - One interface can extend another interface which in turn extends one or more interfaces



Multilevel Inheritance

- **Hierarchical Inheritance**
  - One class is extended by multiple classes
  - One interface is extended by multiple interfaces

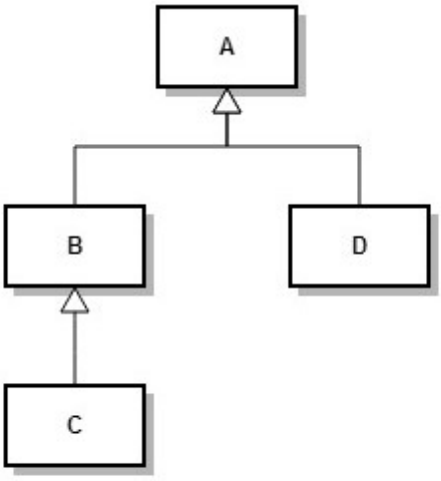
- One interface is implemented by multiple classes



Hierarchical Inheritance

◆ **Hybrid Inheritance**

- If an inheritance hierarchy contains more than one type of inheritance mentioned above, it is called Hybrid inheritance



Hybrid Inheritance

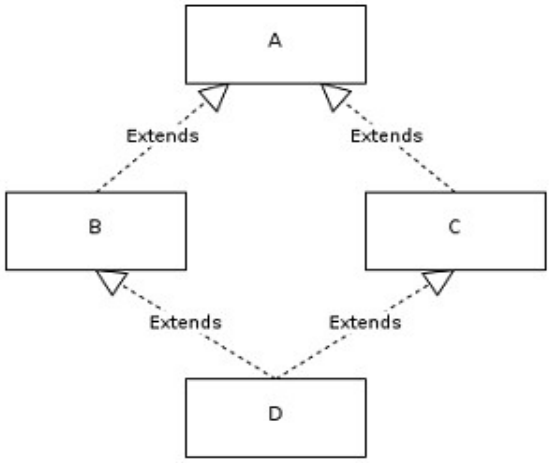
➢ Java supports Single, Multilevel, Hierarchical and Hybrid inheritance by using classes.

➢ Java supports all types of inheritance by using interfaces.

Note: Java doesn't support multiple inheritance by using classes, we will discuss more about this later.

4.1.2.3. Why Java doesn't support Multiple inheritance using classes:

Consider the below scenario where class D is extending two classes B and C where as both B and C are extending class A.



Assume that classes B and C are overriding a method defined in class A, by providing their own implementation. Now D inherits from both B and C doing multiple inheritance. D should inherit that overridden method, which overridden method will be used? Will it be from B or C? We have an ambiguity here. This is the reason why java doesn't support multiple inheritance using classes.

Consider the above scenario for the case where A, B and C are interfaces and D is a class implementing B and C. As interface doesn't have any implementation, even if both B and C are having the same methods, D has to provide implementation for only one method and it has to be implemented in D class. Hence there is no ambiguity. This is the reason why java supports multiple inheritance using interfaces.

4.1.2.4. Super Keyword:

- Super keyword is used to access the variables/methods of the object of super class. This can be done by using the super keyword followed by '.' operator followed by the respective variable or method. This form of super is used when the variable/method used in super class is re-declared in sub class (ie. variable is hidden or method is overridden in sub class).

- Super keyword is also used to refer to the constructor of super class from the constructor of sub class. Ex: super();
- Whenever we try to create the object of sub class, the constructor of sub class will internally invoke the empty constructor of base class. If we want to invoke a parametrized constructor of base class, we need to use super() method and give the respective parameters to this method.

Let us go through the following example for usage of super.

```
class Employee {
        int empId;
        String empName;
        String designation;
        Employee() {                    // Employee Constructor
                System.out.println("Employee Constructor");
        }
        public void display(){
                System.out.println("Employee display method");
        }
}
class RegularEmployee extends Employee {
        int basic;
        RegularEmployee(){
                System.out.println("Regular Employee Constructor");
        }
        public void display(){
                System.out.println("RegularEmployee display method");
        }
        public void abc(){
                display();
                super.display();
        }
        public static void main(String[] s){
                RegularEmployee re = new RegularEmployee(); //line 1
                re.abc();       //line 2
        }
}
```

The output of above program when line 1 is executed would be:

Employee Constructor

Regular Employee Constructor

The output of above program when line 2 is executed would be:

In RegularEmployee display method

In Employee display method

When RegularEmployee class is compiled, an extra line super() will be added by default as it extends the Employee class. Hence in line 1, first the constructor of Employee will be invoked followed by the constructor of RegularEmployee. Always the order of invocation of constructors is from base class to subclass.

In RegularEmployee class, the display() method is re-declared (overridden). When we try to invoke the display method on RegularEmployee object, always the display method in RegularEmployee would be invoked. If we try to invoke the display method of Employee, we need to use super.display() in RegularEmployee class. The output for line 2 clarifies this concept.

- super() is used to invoke the base class(immediate parent class) constructor where as this() is used to invoke the constructor of same class.
- Similar to the way that this() should be the first statement inside the constructor, super() should also be the first statement inside the sub class constructor. Hence super() and this() both cannot be used in the same constructor.
- If a constructor does not explicitly invoke a superclass constructor, the Java compiler automatically inserts a call to the no-argument constructor of the superclass. If the super class does not have a no-argument constructor, we will get a compile-time error. Object does have such a constructor, so if Object is the only superclass, there is no problem.
- If a subclass constructor invokes a constructor of its superclass, either explicitly or implicitly, there will be a whole chain of constructors called, all the way back to the constructor of java.lang.Object class. This is called constructor chaining.

4.1.2.5. Instance of operator:

- The instanceof operator compares an object to a specified type. You can use it to test if an object is an instance of a class, an instance of a subclass, or an instance of a class that implements a particular interface.

- All objects are by default instance of java.lang.Object as every class implicitly extends java.lang.object.
- null is not an instance of any class or interface.

The following program, InstanceofDemo, defines a parent class (named Parent), a simple interface (named MyInterface), and a child class (named Child) that inherits from the parent and implements the interface.

```
class Parent {}
interface MyInterface {}
class Child extends Parent implements MyInterface {}
class InstanceofDemo {
        public static void main(String[] args) {
        Parent obj1 = new Parent();
        Parent obj2 = new Child();
         System.out.println("obj1 instanceof Parent: "
                        + (obj1 instanceof Parent));
         System.out.println("obj1 instanceof Child: "
                        + (obj1 instanceof Child));
        System.out.println("obj1 instanceof MyInterface: "
                        + (obj1 instanceof MyInterface));
        System.out.println("obj2 instanceof Parent: "
                        + (obj2 instanceof Parent));
        System.out.println("obj2 instanceof Child: "
                        + (obj2 instanceof Child));
        System.out.println("obj2 instanceof MyInterface: "
                                + (obj2 instanceof MyInterface));
        System.out.println("obj2 instanceof Object: "
                                        + (obj2 instanceof Object));
        }
}
```

Output:

obj1 instanceof Parent: true

obj1 instanceof Child: false

obj1 instanceof MyInterface: false

obj2 instanceof Parent: true

obj2 instanceof Child: true

obj2 instanceof MyInterface: true

obj2 instanceof Object: true

4.1.3. Pros & Cons

Pros:

1. Inheritance gives us a way to reuse the existing code and also to specialize the behavior depending on need.
2. It is easier to add new subclasses using inheritance because inheritance comes with polymorphism. If you have a bit of code that relies only on a superclass interface, that code can work with a new subclass without change.
3. Inheritance's single invocation of an inherited superclass method implementation gives more code readable,good performance most often compared to explicit method-invocation forwarding (or delegation) approach of composition.

Cons:

1. With inheritance, you get the image of the superclass in your subclass object image as soon as the subclass is created, and it remains part of the subclass object throughout the lifetime of the subclass.
2. In an inheritance relationship, superclasses are often said to be "fragile," because one little change to a superclass can ripple out and require changes in many other places in the application's code.
3. Inheritance is also sometimes said to provide "weak encapsulation," because if we have code that directly uses a subclass, that code can be broken by changes to a superclass. One of the ways to look at inheritance is that it allows subclass code to reuse superclass code. For example, if subclass doesn't override a method defined in its superclass, the sub class is in a sense reusing super class implementation of the method.

4.1.4. Syntax

- Inheritance can be done in Java by using ***extends*** and ***implements*** keywords
- The ***extends*** keyword is used to inherit:

  *class is extending from an abstract class, it can declared as abstract or all concrete methods can be defined.

-

a) a class by another class and

b) an interface by another interface

Let us consider the class Employee :

```
class Employee {
        int empId;
        String empName;
        String designation;
}
```

There are two types of employees – Regular and Contract. Both the type of employees are having the properties mentioned in Employee above and additionally some specific properties. In this case, instead of declaring these common properties, we will reuse them by extending the Employee class ie.

```
class RegularEmployee extends Employee {
        int basic;
}

class ContractEmployee extends Employee {
        int hourlyRate;
        double numHours;
}
```

Note: A class can extend only one another class in Java.

Similarly an interface can inherit another interface by using extends keyword:

```
interface I1 {
        void display();
}
interface I2  extends I1{
        void add();
}
```

Note: An interface can extend any number of interfaces in Java.

- The ***implements*** keyword is used to inherit an interface by a class

```
class c1 implements I1{
        public void display(){
                // Code here
        }
}
```

Note: 1) An class can implement any number of interfaces in Java.

2) An interface cannot extend or implement a class

- Following table summarizes the usage of **extends** and **implements** keywords:

	Class	**Interface**
Class	extends one	implements (1 to many)
Interface	----------	extends (1 to many)

4.2. Code Snippets

4.3. Real Time Usage

1) When there is a related hierarchy of classes or interfaces, instead of duplicating the methods/variables we reuse the common aspects by putting it in base class/interface and let the sub classes extend the base class/interface and also add new methods/variables. Inheritance is used mainly for code reuse.

4.4. Check Yourself

1) Java supports multiple inheritance with the help of _____

Ans. Interface

2) super keyword is used to refer to the constructor of _____ from the constructor of _____

Ans. Parent class, Child class

3) _____ keyword is used to access the members of parent class.

Ans. super

4) _____ keyword is used to inherit an interface by a class

Ans. implements

5) instanceof operator returns a _____ value:

a) int b)boolean c) String d)character

Ans.. b)

5. Error Zone - commonly made mistakes

The readability and re-usability of code can also be obtained by using Composition instead of Inheritance. Inheritance needs to be used only when there is an is-a relationship and whether that is-a relationship will be constant throughout the lifetime of the application.

6. Sample code

7. Additional Information (Optional)

The Object class, in the java.lang package, sits at the top of the class hierarchy tree. Every class is a descendant, direct or indirect, of the Object class. Every class you use or write inherits the instance methods of Object. You need not use any of these methods, but, if you choose to do so, you may need to override them with code that is specific to your class. The methods inherited from Object that are discussed in this section are:

•protected Object clone() throws CloneNotSupportedException

Creates and returns a copy of this object.

•public boolean equals(Object obj)

Indicates whether some other object is "equal to" this one.

•protected void finalize() throws Throwable

Called by the garbage collector on an object when garbage

collection determines that there are no more references to the object

•public final Class getClass()

Returns the runtime class of an object.

•public int hashCode()

Returns a hash code value for the object.

•public String toString()

Returns a string representation of the object.

The notify, notifyAll, and wait methods of Object all play a part in synchronizing the activities of independently running threads in a program, and won't be covered here. There are five of these methods:

•public final void notify()

•public final void notifyAll()

•public final void wait()

•public final void wait(long timeout)

•public final void wait(long timeout, int nanos)

8. Best Practices

Before using inheritance, we need to ensure that we have an is-a relationship and whether that is-a relationship will be constant throughout the lifetime of the application .

9. Summary or Take Away Points

- Inheritance is the process of acquiring the properties of one class by another. It allows us to build hierarchies of classes/interfaces. It is also called IS-A relationship between classes/interfaces involved in the hierarchy
- A subclass inherits all of the public and protected members of its parent, irrespective of the package the subclass is in. If the subclass is in the same package as its parent, it also inherits the default members of the parent. Private members of a class cannot be inherited.
- There are different types of inheritance - Single, Multiple, Multilevel, Hierarchical and Hybrid
- Java supports Single, Multilevel, Hierarchical and Hybrid inheritance by using classes.
- Java supports all types of inheritance by using interfaces.
- If we don't want our class to be extensible, the final keyword needs to be used while declaring the class.
- Super keyword is used to access the variables/methods of the object of immediate super class.
- Super keyword is also used to refer to the constructor of super class from the constructor of sub class.

- The instanceof operator compares an object to a specified type. You can use it to test if an object is an instance of a class, an instance of a subclass, or an instance of a class that implements a particular interface.

10. Related Topics:

Class relationships, Interface, Abstract class and Final class.

11. References

http://docs.oracle.com/javase/tutorial/java/IandI/summaryinherit.htm